

Heart Disease Risk Prediction MLOps Pipeline

1. Project Overview

This project implements an end-to-end MLOps workflow for predicting the risk of heart disease from patient clinical data. The solution uses the UCI Cleveland Heart Disease dataset and covers data acquisition, EDA, preprocessing, model training, experiment tracking, reproducible model packaging, API serving, containerization, CI/CD, Kubernetes deployment, and monitoring.

The final application exposes a FastAPI prediction API that accepts patient features as JSON and returns a binary heart disease prediction with a confidence score.

Repository: <https://github.com/2024CS05012/MLOps-Heart-Disease>

2. Setup and Installation

The complete setup and execution instructions are provided in:

- [docs/SETUP.md](#)

Main setup commands:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

The project can be verified locally using Python, Docker Desktop, Docker Compose, and Docker Desktop Kubernetes.

3. Dataset

Dataset: Heart Disease UCI Dataset, Cleveland processed dataset.

The dataset is downloaded using:

```
python data/raw/download_dataset.py
```

The original UCI target represents disease severity from 0 to 4. For this binary classification problem, target value 0 is treated as no heart disease and values 1 to 4 are mapped to heart disease.

Final features include age, sex, chest pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise-induced angina, ST depression, slope, number of major vessels, and thalassemia category.

4. Exploratory Data Analysis

EDA artifacts are generated with:

```
python -m src.data.generate_eda_artifacts
```

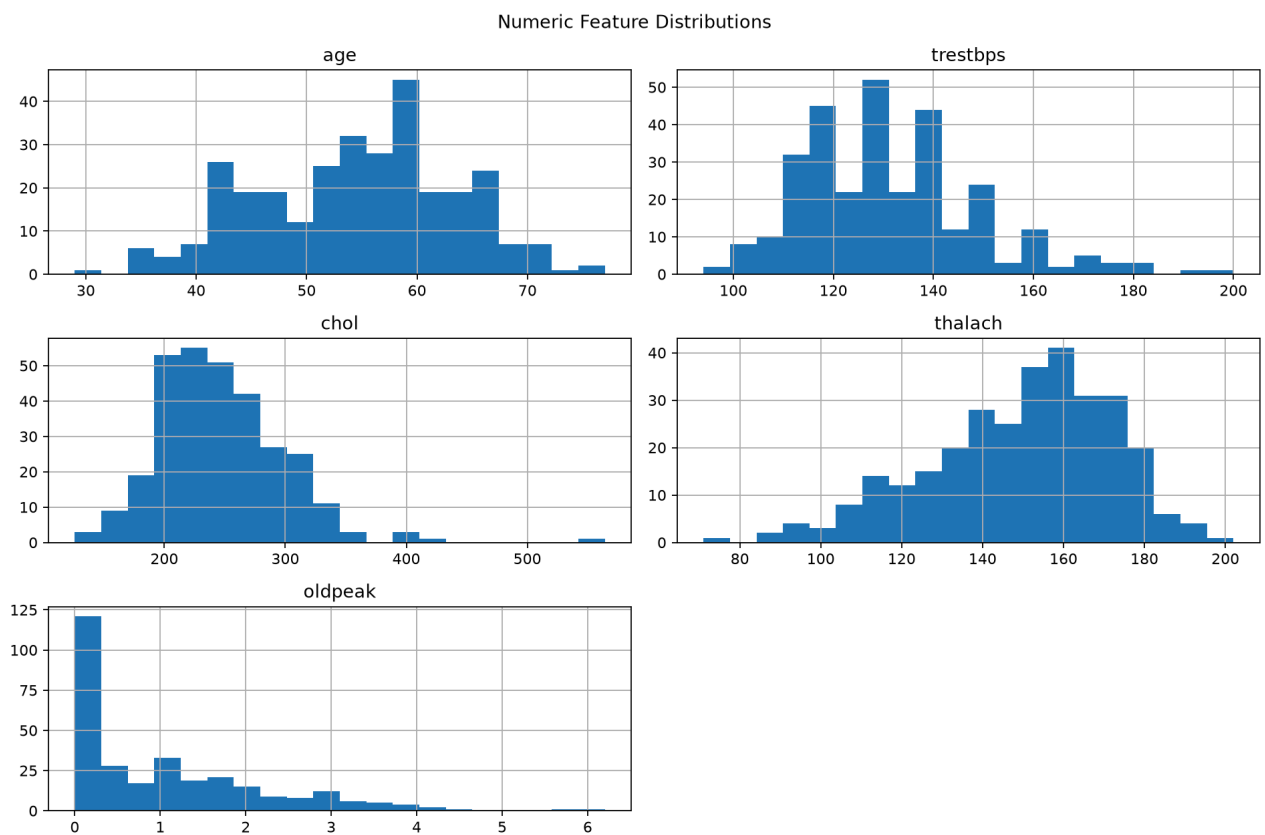
The EDA includes:

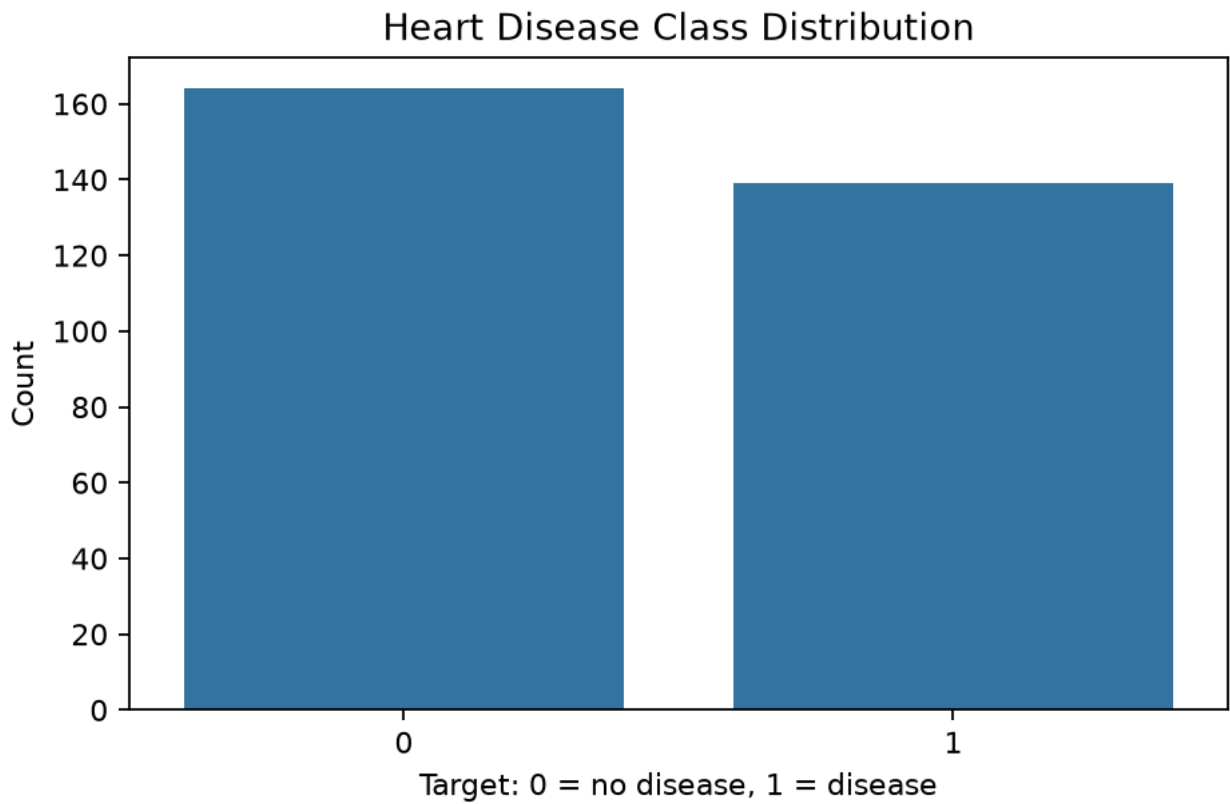
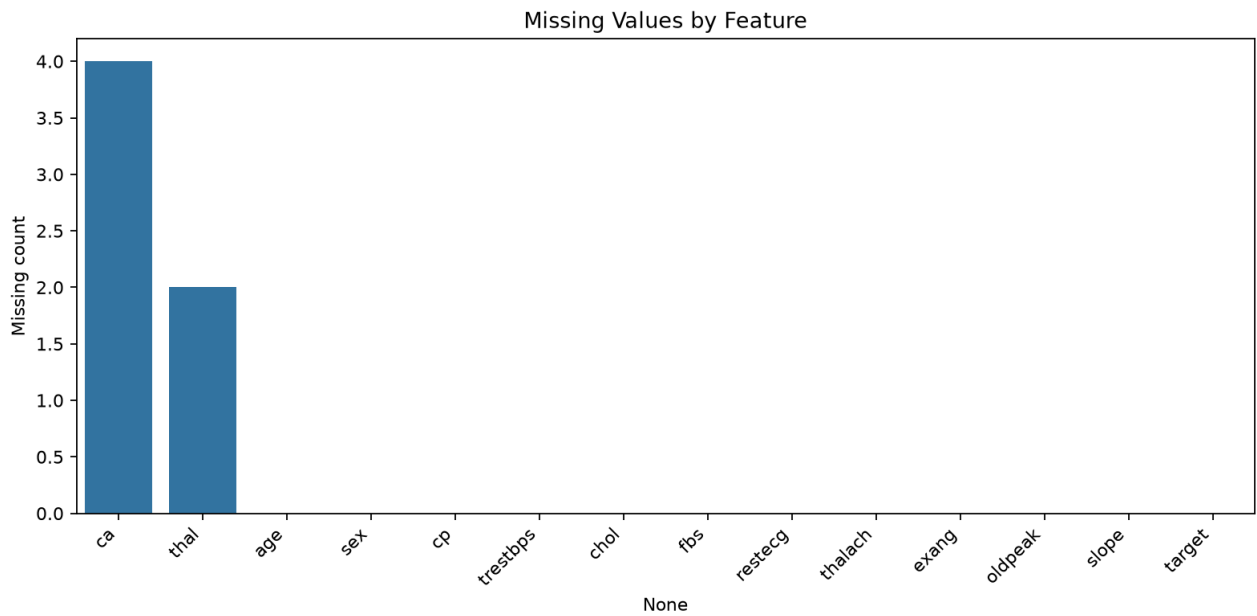
- Numeric feature histograms
- Missing-value analysis
- Class distribution
- Correlation heatmap
- Maximum heart rate relationship by target class

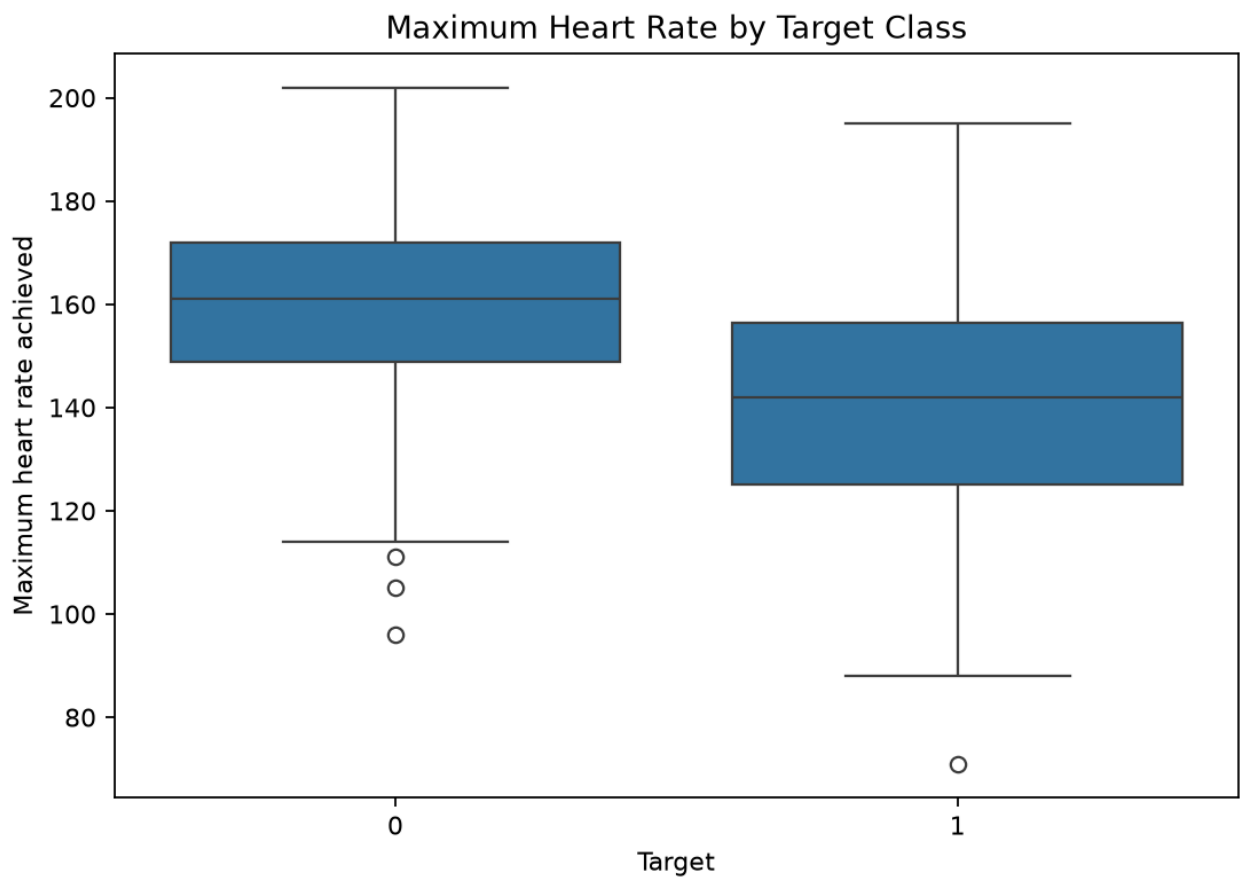
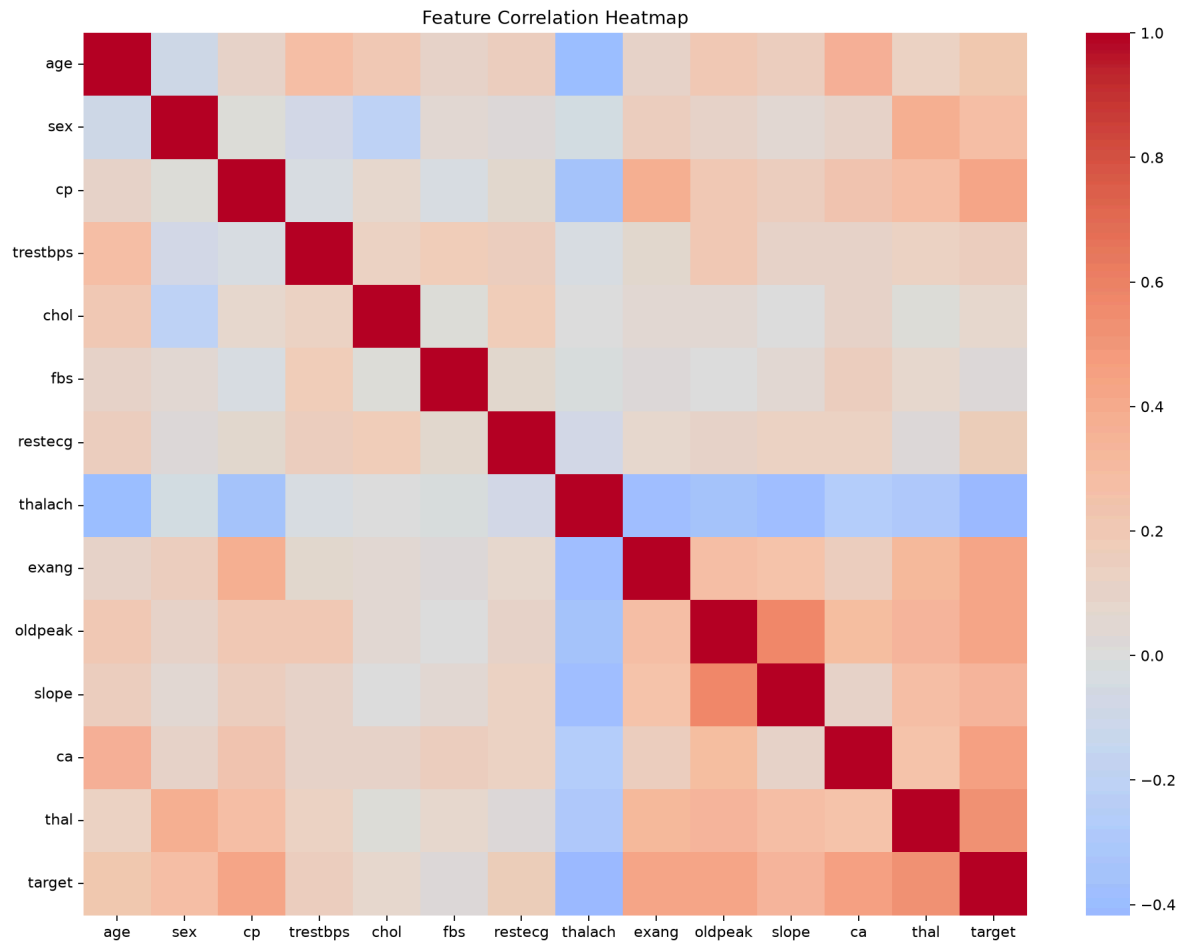
Generated EDA files:

- [artifacts/eda/feature_histograms.png](#)
- [artifacts/eda/missing_values.png](#)
- [artifacts/eda/class_distribution.png](#)
- [artifacts/eda/correlation_heatmap.png](#)
- [artifacts/eda/thalach_by_target.png](#)

Captured screenshots:







5. Preprocessing and Feature Engineering

The project uses a reusable sklearn preprocessing pipeline defined in `src/features/engineering.py`.

Preprocessing choices:

- Missing numeric values are imputed using the median.
- Numeric features are scaled with `StandardScaler`.
- Missing categorical values are imputed using the most frequent value.
- Categorical features are encoded with `OneHotEncoder`.
- Preprocessing is stored inside the final sklearn `Pipeline`, so training and inference use the same transformations.

This satisfies reproducibility because the saved model artifact includes both feature transformations and the trained classifier.

6. Model Development

Two classification models are trained:

- Logistic Regression
- Random Forest

Training is implemented in `src/models/train.py`.

Training command:

```
python -m src.models.train
```

Model selection and tuning:

- Each model is wrapped in an sklearn `Pipeline`.
- Hyperparameter tuning is performed using `GridSearchCV`.
- Stratified cross-validation is used to preserve class balance across folds.
- ROC-AUC is used as the grid-search optimization score.

Evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Cross-validation metric averages

Current generated classification report summary:

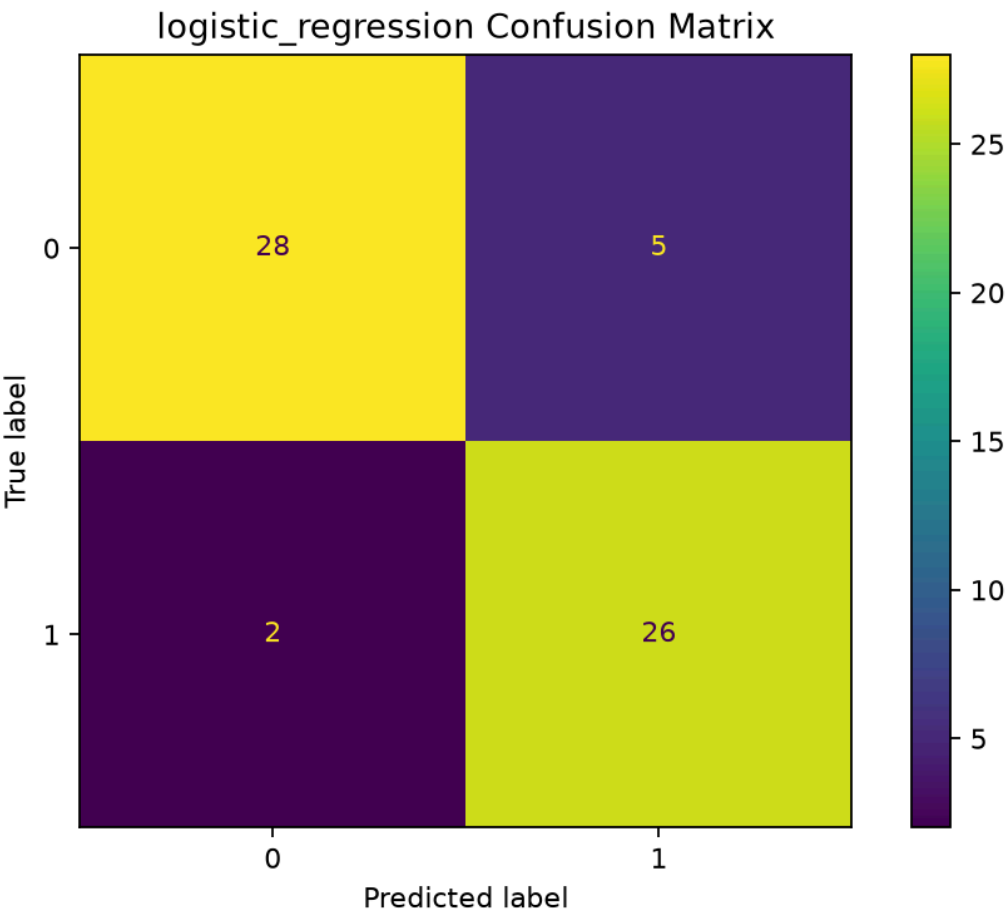
Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1
Logistic Regression	0.8852	0.8899	0.8852	0.8854

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1
Random Forest	0.8852	0.8899	0.8852	0.8854

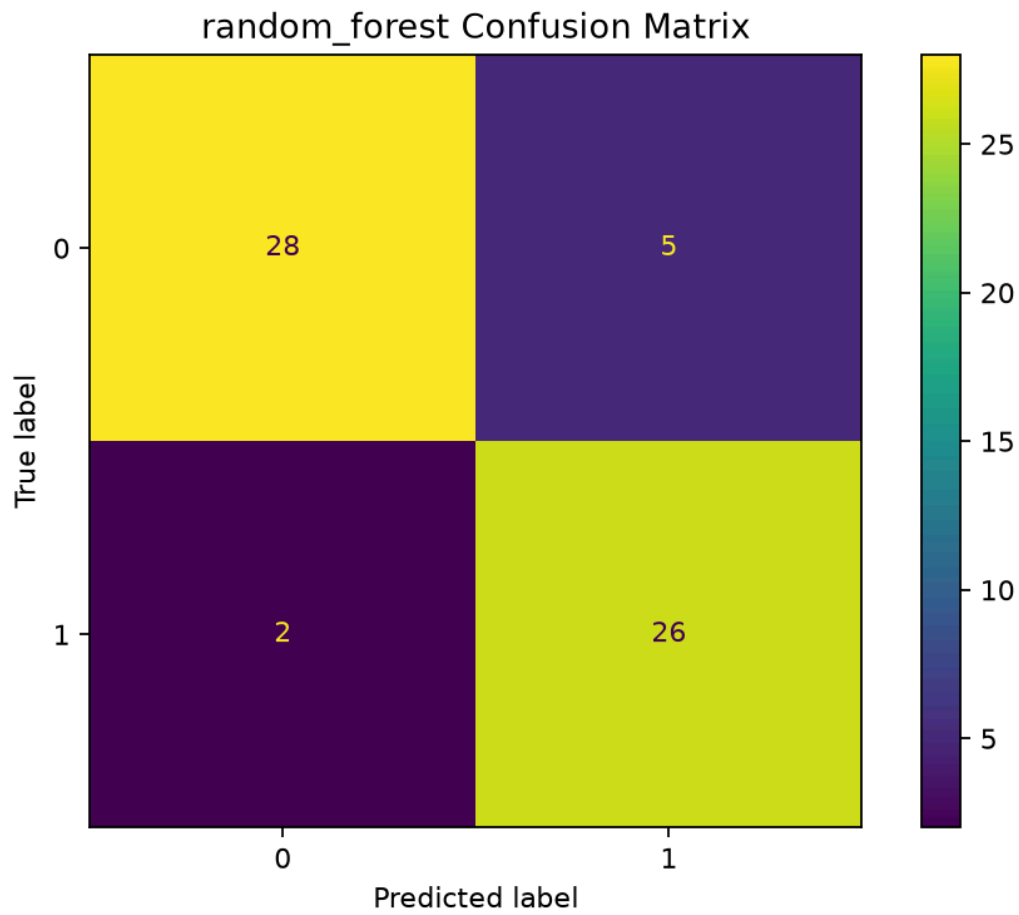
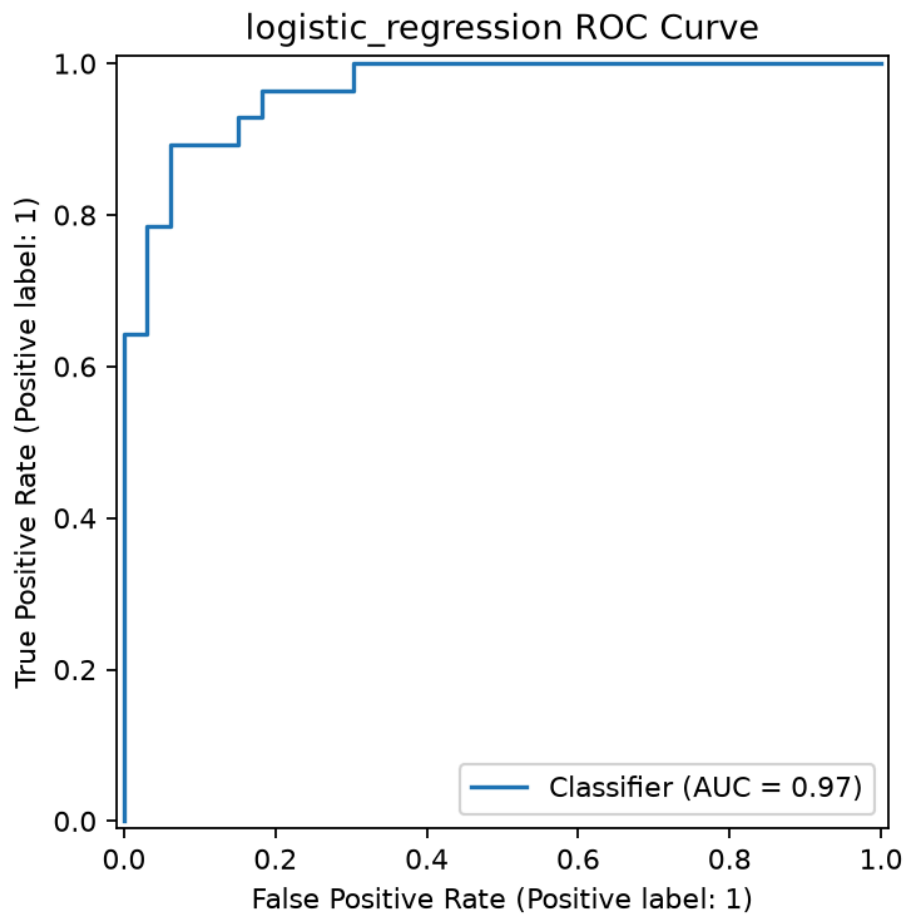
Evaluation artifacts:

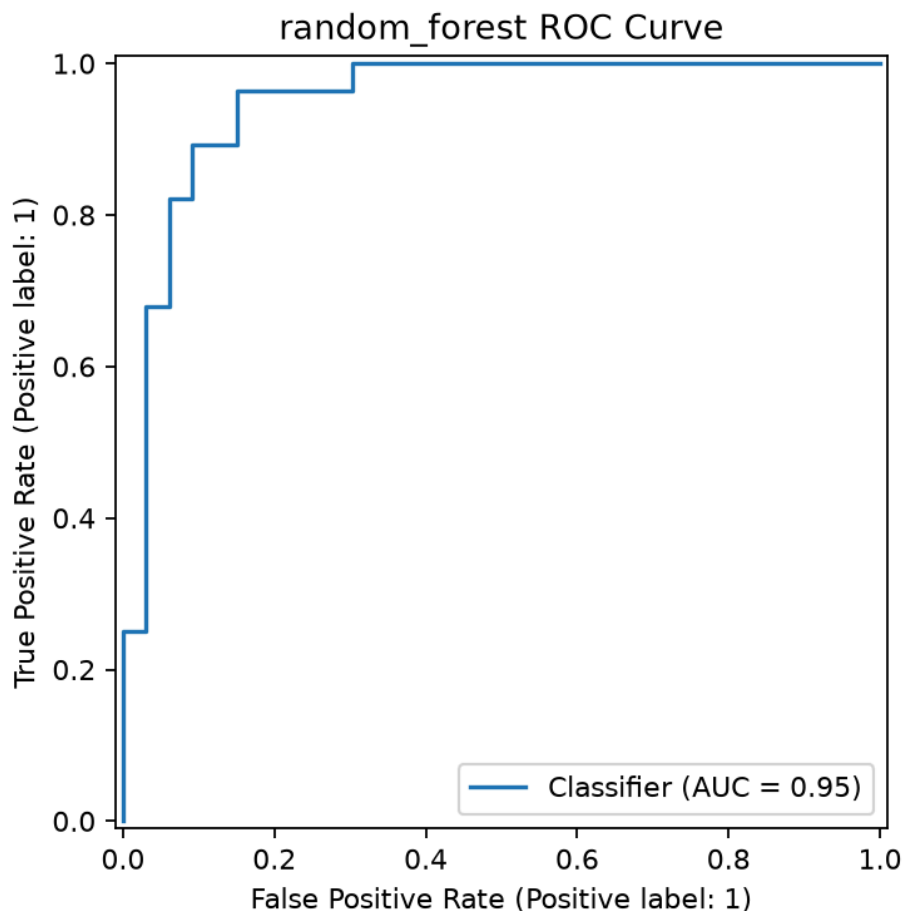
- artifacts/logistic_regression/classification_report.json
- artifacts/logistic_regression/confusion_matrix.png
- artifacts/logistic_regression/roc_curve.png
- artifacts/random_forest/classification_report.json
- artifacts/random_forest/confusion_matrix.png
- artifacts/random_forest/roc_curve.png

Captured screenshots:



•





7. Experiment Tracking

MLflow is used for experiment tracking. The MLflow integration is implemented in [src/models/mlflow_utils.py](#).

MLflow tracks:

- Model name
- Best hyperparameters
- Accuracy, precision, recall, F1-score, ROC-AUC
- Cross-validation metrics
- Confusion matrix images
- ROC curve images
- Classification reports
- Saved joblib model artifacts
- Serialized sklearn model artifacts

Start the MLflow UI with:

```
mlflow ui --backend-store-uri sqlite:///mlruns/mlflow.db
```

Then open:

- <http://127.0.0.1:5000>

Captured screenshots:

The first screenshot shows the mlflow web interface for the 'heart-disease-ml' experiment. The 'Training runs' table lists 22 matching runs. The table columns are Run Name, Created, Duration, Source, and Models. The runs are sorted by Created time, showing a mix of 'random_forest' and 'logistic_regression' models trained using 'pytest' and 'train.py' sources.

The second screenshot shows the details for a specific 'random_forest' model. The 'Overview' tab is selected, displaying the model's description, metrics, and parameters. The 'Metrics (11)' table shows various performance metrics, and the 'Parameters (4)' table shows the model's configuration.

8. Model Packaging and Reproducibility

The final models are saved in reusable joblib format:

- `models/logistic_regression.joblib`
- `models/random_forest.joblib`

The saved artifacts are sklearn pipelines, so they include both preprocessing and the trained classifier. This ensures the API uses the same preprocessing logic that was used during model training.

Reproducibility files:

- `requirements.txt`
- `src/features/engineering.py`
- `src/models/train.py`
- `models/*.joblib`
- `docs/SETUP.md`

9. API Serving

The model is served using FastAPI in `src/api/main.py`.

Endpoints:

- `GET /health`
- `POST /predict`
- `GET /metrics`

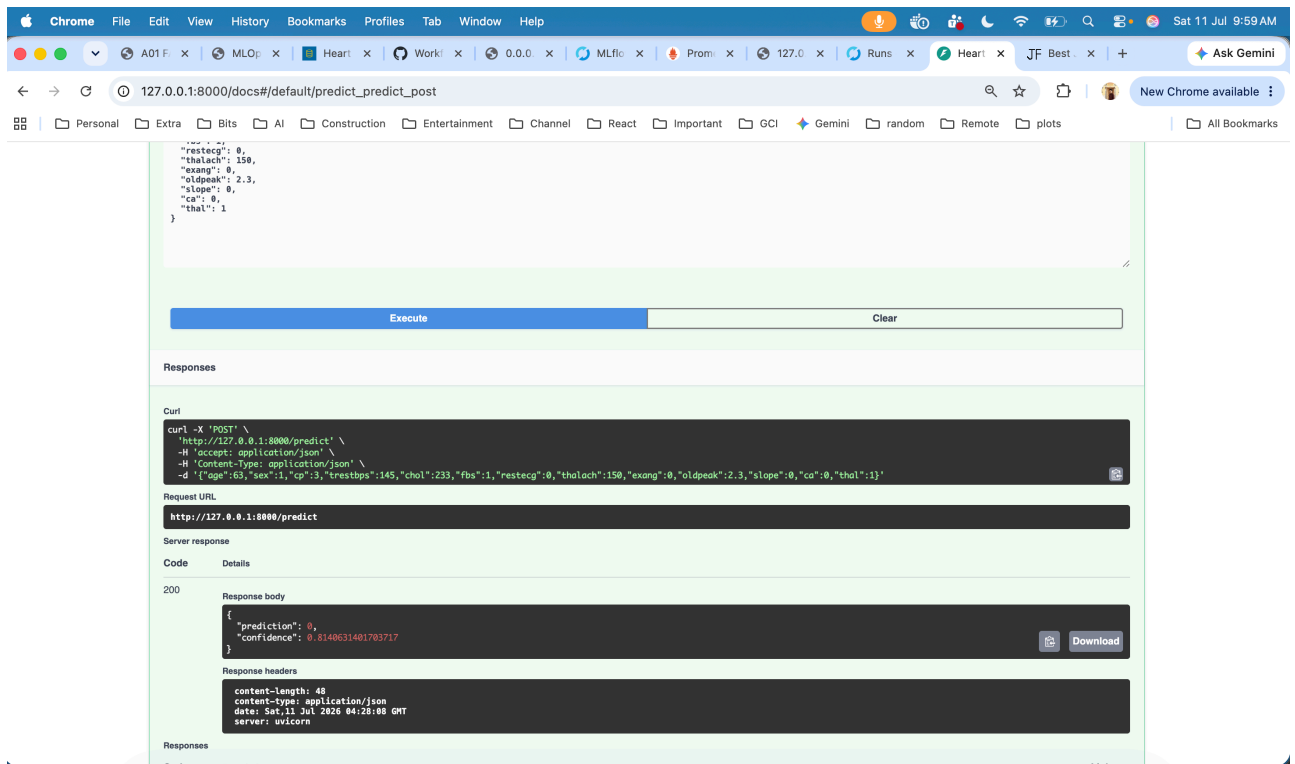
The `/predict` endpoint accepts JSON input and returns:

- `prediction`: binary class prediction
- `confidence`: maximum predicted class probability

Sample request:

```
curl -X POST http://127.0.0.1:8000/predict \  
-H "Content-Type: application/json" \  
-d \  
'{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,"fbs":1,"restecg":0,"th \  
alach":150,"exang":0,"oldpeak":2.3,"slope":0,"ca":0,"thal":1}'
```

Captured screenshot:



10. Containerization

The API is containerized using Docker.

Build and run:

```
docker build -f docker/Dockerfile -t heart-disease-api:local .
docker run --rm -p 8000:8000 --name heart-disease-api heart-disease-api:local
```

Verify:

```
curl http://127.0.0.1:8000/health
```

Captured screenshots:

Code File Edit Selection View Go Run Terminal Window Help

MLOps

FINAL_REPORT.md M x SETUP.py ... Preview FINAL_REPORT.md x

docs > FINAL_REPORT.md > abc # Heart Disease

1 # Heart Disease Risk

heart-disease-api

heart-disease-api:local

Verify:

curl http://127.0.0.1:8000/health

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(.venv) shyamgupta@Shyams-MacBook-Air MLOps % s

(.venv) shyamgupta@Shyams-MacBook-Air MLOps % clear

(.venv) shyamgupta@Shyams-MacBook-Air MLOps % docker images

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	In Use
docker-api:latest	b1bed8bb2161	1.38GB	301MB	U	
docker/desktop-cloud-provider-kind:v0.6.0	46d51b990e97	587MB	156MB	U	
docker/desktop-containerd-registry-mirror:v0.0.4	1de142695bb0	51.1MB	16.3MB	U	
envoyproxy/envoy:v1.36.7	29d778ba078e	239MB	60.5MB	U	
heart-disease-api:local	30e9c971a378	1.38GB	301MB	U	
kindest/node:v1.36.1	3489c7674813	1.27GB	350MB	U	
prom/prometheus:v2.54.1	f6639335d34a	374MB	102MB	U	

(.venv) shyamgupta@Shyams-MacBook-Air MLOps % docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
c606393ff6a9	heart-disease-api:local	"uvicorn src.api.mai..."	2 hours ago	Up 2 hours	0.0.0.0:18000->8000/tcp
, [::]:18000->8000/tcp	heart-disease-api				
bebb667c1701	docker-api	"uvicorn src.api.mai..."	3 hours ago	Up 3 hours	0.0.0.0:8000->8000/tcp,
[::]:8000->8000/tcp	docker-api-1				
f55e3565c4f1	prom/prometheus:v2.54.1	"/bin/prometheus --c..."	3 hours ago	Up 3 hours	0.0.0.0:9090->9090/tcp,
[::]:9090->9090/tcp	docker-prometheus-1				

(.venv) shyamgupta@Shyams-MacBook-Air MLOps %

4 Hidden T...

Code File Edit Selection View Go Run Terminal Window Help

MLOps

FINAL_REPORT.md M x SETUP.py ... Preview FINAL_REPORT.md x

docs > FINAL_REPORT.md > abc # Heart Disease

1 # Heart Disease Risk

heart-disease-api

heart-disease-api:local

Verify:

curl http://127.0.0.1:8000/health

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(.venv) shyamgupta@Shyams-MacBook-Air MLOps % curl http://127.0.0.1:8000/health

{\"status\":\"ok\"}

(.venv) shyamgupta@Shyams-MacBook-Air MLOps %

4

4 Hidden T...

11. CI/CD Pipeline

GitHub Actions is configured in [.github/workflows/ci.yml](#).

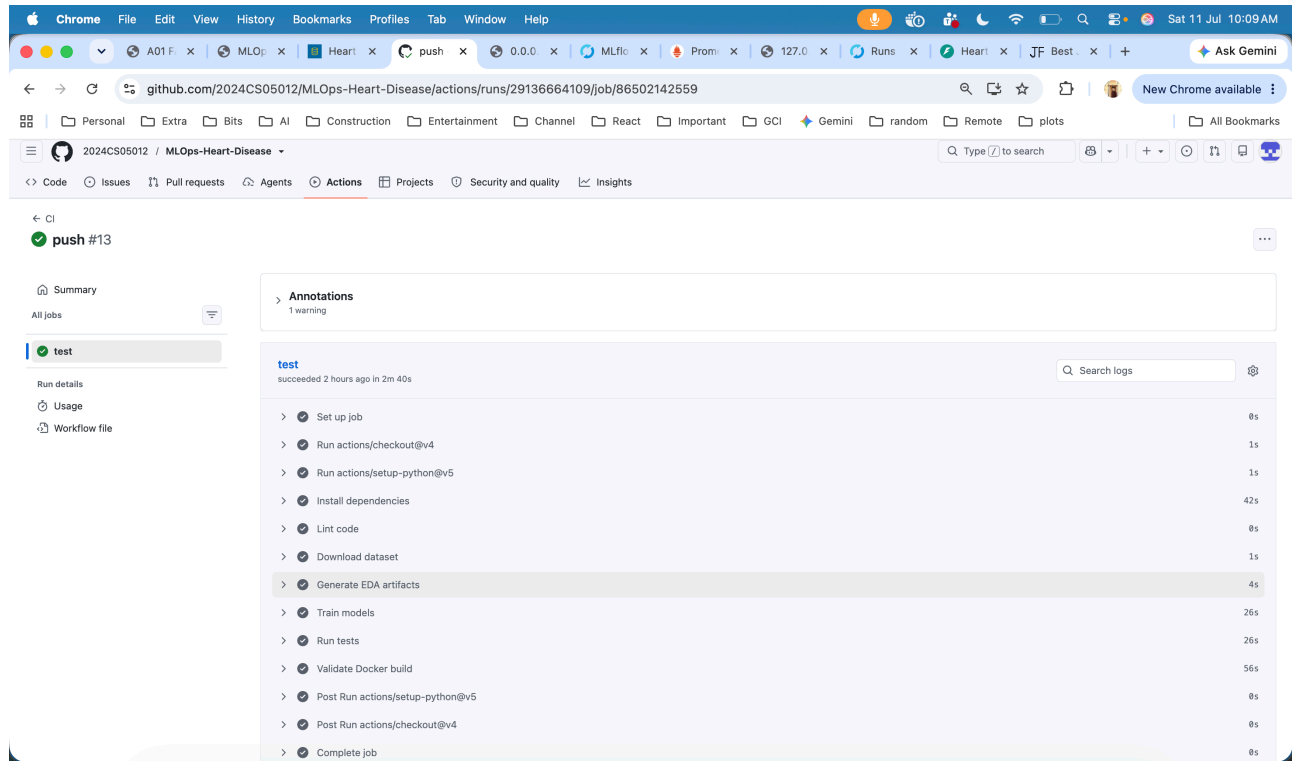
The workflow runs on push and pull request. It performs:

- Dependency installation
- Ruff linting
- Dataset download
- EDA artifact generation
- Model training

- Pytest test suite
- Docker build validation

This ensures the pipeline fails clearly if code quality checks, tests, training, or Docker build validation fail.

Captured screenshot:



12. Kubernetes Deployment

Kubernetes deployment is defined in `k8s/deployment.yaml`.

The manifest includes:

- Deployment for the FastAPI container
- Readiness probe on `/health`
- Liveness probe on `/health`
- LoadBalancer service exposing port 8000

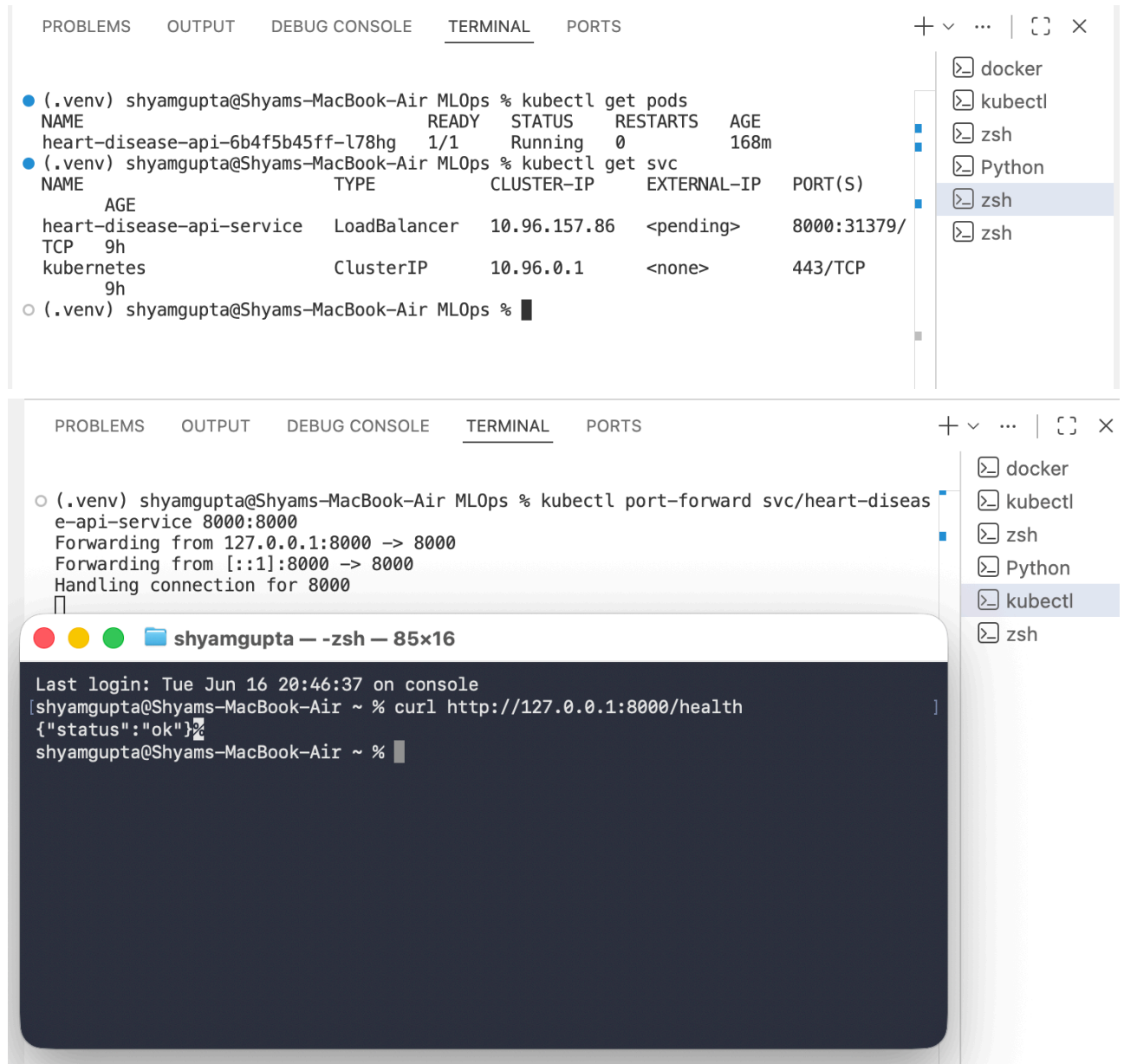
Docker Desktop Kubernetes deployment commands:

```
docker build -f docker/Dockerfile -t heart-disease-api:local .
kubectl apply -f k8s/deployment.yaml
kubectl rollout status deployment/heart-disease-api
kubectl get pods
kubectl get svc heart-disease-api-service
kubectl port-forward svc/heart-disease-api-service 18000:8000
```

Verify:

```
curl http://127.0.0.1:18000/health
```

Captured screenshots:



13. Monitoring and Logging

Monitoring and logging are implemented with FastAPI request logging and Prometheus-compatible metrics.

Implemented monitoring/logging:

- API request logs include method, path, status code, and duration.
- `/metrics` exposes Prometheus metrics.
- `monitoring/prometheus.yml` configures Prometheus to scrape the API service.
- `docker/docker-compose.yml` starts the API and Prometheus together.

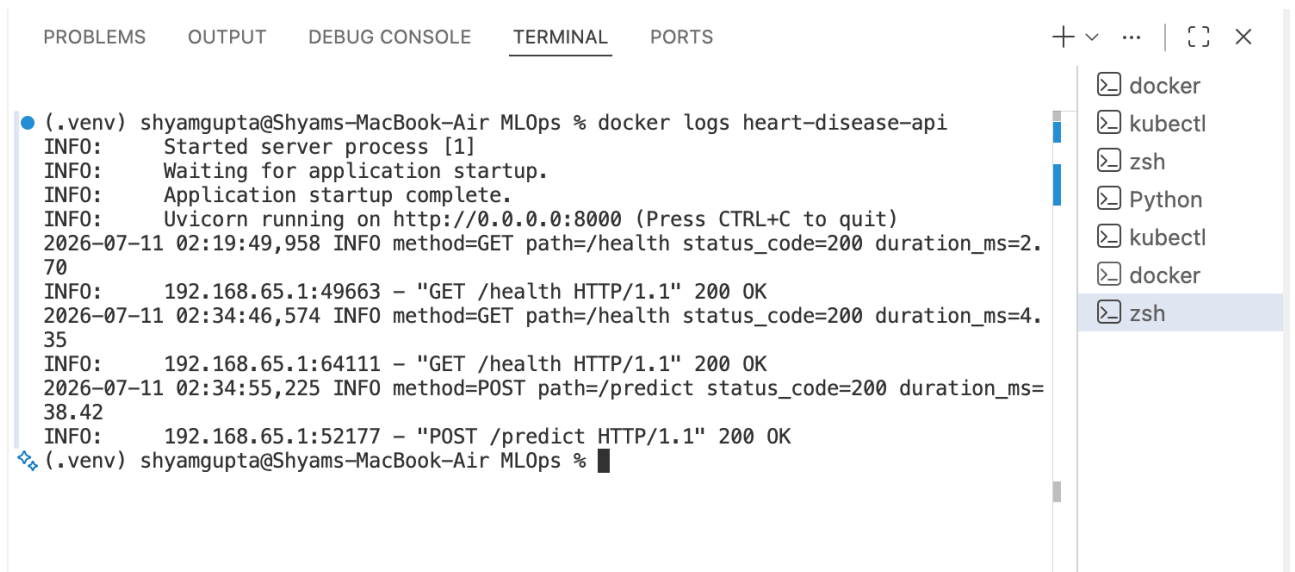
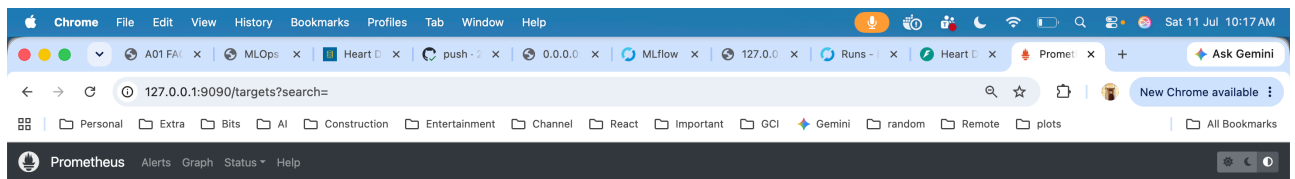
Run:

```
docker compose -f docker/docker-compose.yml up -d --build
```

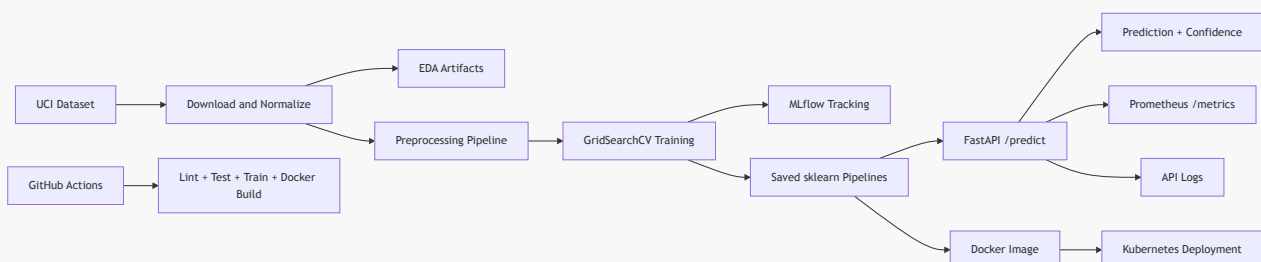
Verify metrics:

```
curl http://127.0.0.1:8000/metrics
curl http://127.0.0.1:9090/api/v1/targets
```

Captured screenshots:



14. Architecture



15. Final Submission Checklist

- Code repository link included.
- Setup instructions included in [docs/SETUP.md](#).
- EDA artifacts generated.
- Two tuned models trained and evaluated.
- MLflow runs logged.
- Models saved as reusable sklearn pipelines.
- Unit tests and CI/CD workflow included.
- Docker container builds and serves the API locally.
- Kubernetes deployment manifest included and tested locally.
- Monitoring through API logs and Prometheus metrics included.
- Screenshots added under [screenshots/](#).
- Final Markdown report exported to PDF if required by the course portal.